11.3 ASSIGNMENT

A.Krishna Koushik

2303a51589

Task 1: Smart Contact Manager (Arrays & Linked Lists)
Scenario

SR University's student club requires a simple Contact Manager Application to store members' names and phone numbers. The system should support efficient addition, searching, and deletion of contacts.

CODE:

```
class ContactManagerArray:
    def __init__(self):
        self.contacts = []

    def add_contact(self, name, phone):
        """Add a contact to the list"""
        self.contacts.append({"name": name, "phone": phone})
        print(f"Contact {name} added.")

    def search_contact(self, name):
        """Search for a contact by name"""
        for contact in self.contacts:
            if contact["name"].lower() == name.lower():
                return contact
        return None

    def delete_contact(self, name):
        """Delete a contact by name"""
        for i, contact in enumerate(self.contacts):
            if contact["name"].lower() == name.lower():
                self.contacts.pop(i)
                print(f"Contact {name} deleted.")
                return True
```

```
        print(f"Contact {name} not found.")
        return False

    def display_contacts(self):
        """Display all contacts"""
        for contact in self.contacts:
            print(f"{contact['name']}: {contact['phone']}")

# Contact Manager - Linked List Implementation
class Node:
    def __init__(self, name, phone):
        self.data = {"name": name, "phone": phone}
        self.next = None

class ContactManagerLinkedList:
    def __init__(self):
        self.head = None

    def add_contact(self, name, phone):
        """Add a contact to the linked list"""
        new_node = Node(name, phone)
        if not self.head:
            self.head = new_node
        else:
            current = self.head
            while current.next:
                current = current.next
            current.next = new_node
        print(f"Contact {name} added.")

    def search_contact(self, name):
```

```
    """Search for a contact by name"""
    current = self.head
    while current:
        if current.data["name"].lower() == name.lower():
            return current.data
        current = current.next
    return None

def delete_contact(self, name):
    """Delete a contact by name"""
    if not self.head:
        print(f"Contact {name} not found.")
        return False

    if self.head.data["name"].lower() == name.lower():
        self.head = self.head.next
        print(f"Contact {name} deleted.")
        return True

    current = self.head
    while current.next:
        if current.next.data["name"].lower() == name.lower():
            current.next = current.next.next
            print(f"Contact {name} deleted.")
            return True
        current = current.next

    print(f"Contact {name} not found.")
    return False

def display_contacts(self):
```

```
        """Display all contacts"""
        current = self.head
        while current:
            print(f"{current.data['name']}: {current.data['phone']}")
            current = current.next

# Comparison and Demo
if __name__ == "__main__":
    print("=== Array-based Contact Manager ===")
    arr_manager = ContactManagerArray()
    arr_manager.add_contact("Alice", "123-4567")
    arr_manager.add_contact("Bob", "234-5678")
    arr_manager.search_contact("Alice")
    arr_manager.delete_contact("Bob")
    arr_manager.display_contacts()

    print("\n=== Linked List-based Contact Manager ===")
    ll_manager = ContactManagerLinkedList()
    ll_manager.add_contact("Charlie", "345-6789")
    ll_manager.add_contact("Diana", "456-7890")
    ll_manager.search_contact("Diana")
    ll_manager.delete_contact("Charlie")
    ll_manager.display_contacts()

    print("\n=== Performance Comparison ===")
    print("INSERTION:")
    print("  Array: O(1) amortized")
    print("  Linked List: O(n) - requires traversal to end")
    print("\nDELETION:")
    print("  Array: O(n) - requires shift after removal")
    print("  Linked List: O(n) - requires traversal + pointer update")
```

```
print("\nSEARCH:")
print("  Both: O(n)")
```

OUTPUT:

```
=== Array-based Contact Manager ===
Contact Alice added.
Contact Bob added.
Contact Bob deleted.
Alice: 123-4567
Contact Bob deleted.
Alice: 123-4567
Alice: 123-4567

=== Linked List-based Contact Manager ===
=== Linked List-based Contact Manager ===
Contact Charlie added.
Contact Charlie added.
Contact Diana added.
Contact Diana added.
Contact Charlie deleted.
Contact Charlie deleted.
Diana: 456-7890
Diana: 456-7890


=== Performance Comparison ===
=== Performance Comparison ===
INSERTION:
  Array: O(1) amortized
  Linked List: O(n) - requires traversal to end

DELETION:
  Array: O(n) - requires shift after removal
  Linked List: O(n) - requires traversal + pointer update

SEARCH:
  Both: O(n)
```

Observation

The Smart Contact Manager was implemented using both **array (list)** and **linked list** data structures to manage contacts efficiently. Operations such as adding, searching, and deleting contacts were successfully performed in both implementations.

It was observed that:

- Array implementation provides simple and fast insertion at the end.
- Linked list allows dynamic memory allocation.
- Searching requires sequential traversal in both methods.
- Deletion is more efficient in linked lists since no shifting of elements is required.

TASK 2 : Library Book Search System (Queues & Priority Queues)

Scenario

The SRU Library manages book borrow requests. Students and faculty submit

requests, but faculty requests must be prioritized over student requests.

CODE:

class LibraryQueue:

```
def __init__(self):
    self.queue = []

def enqueue(self, name, role, book):
    """Add request to queue"""
    request = {
        "name": name,
        "role": role,
        "book": book
    }
    self.queue.append(request)
    print(f"Request added: {name} ({role})")

def dequeue(self):
    """Remove first request"""
    if not self.queue:
        print("Queue is empty!")
        return None

    request = self.queue.pop(0)
    print(f"Request processed: {request['name']}")
    return request

def display(self):
    """Display all requests"""
    if not self.queue:
        print("No pending requests.")
        return

    print("\nPending Requests (FIFO):")
    for r in self.queue:
```

```
        print(f"{r['name']} - {r['role']} - {r['book']}")

# ---------------------------------
# Priority Queue (Faculty First)
# ---------------------------------

class LibraryPriorityQueue:

    def __init__(self):
        self.queue = []

    def enqueue(self, name, role, book):
        """Add request with priority"""

        request = {
            "name": name,
            "role": role,
            "book": book
        }

        # Faculty gets higher priority (0), Student gets lower (1)
        if role.lower() == "faculty":
            priority = 0
        else:
            priority = 1

        self.queue.append((priority, request))

        # Sort based on priority
        self.queue.sort(key=lambda x: x[0])
```

```
        print(f"Request added: {name} ({role})")

    def dequeue(self):
        """Remove highest priority request"""

        if not self.queue:
            print("Priority Queue is empty!")
            return None

        priority, request = self.queue.pop(0)

        print(f"Request processed: {request['name']} ({request['role']})")
        return request

    def display(self):
        """Display all requests"""

        if not self.queue:
            print("No pending requests.")
            return

        print("\nPending Requests (Priority Queue):")

        for p, r in self.queue:
            print(f"{r['name']} - {r['role']} - {r['book']}")

# ---------------------------------
# Main Program (Testing)
# ---------------------------------

if __name__ == "__main__":
```

11.3 ASSIGNMENT

```
print("=====================================")
print("   LIBRARY BOOK REQUEST SYSTEM")
print("=====================================")

# -------- Normal Queue --------
print("\n---- Normal Queue (FIFO) ----")

normal_queue = LibraryQueue()

normal_queue.enqueue("Ravi", "Student", "Python Basics")
normal_queue.enqueue("Dr. Kumar", "Faculty", "AI Research")
normal_queue.enqueue("Anita", "Student", "Data Structures")
normal_queue.enqueue("Prof. Rao", "Faculty", "Machine Learning")

normal_queue.display()

print("\nProcessing Requests:")
normal_queue.dequeue()
normal_queue.dequeue()

normal_queue.display()

# -------- Priority Queue --------
print("\n---- Priority Queue (Faculty First) ----")

priority_queue = LibraryPriorityQueue()

priority_queue.enqueue("Ravi", "Student", "Python Basics")
priority_queue.enqueue("Dr. Kumar", "Faculty", "AI Research")
priority_queue.enqueue("Anita", "Student", "Data Structures")
```

```
    priority_queue.enqueue("Prof. Rao", "Faculty", "Machine Learning")


    priority_queue.display()


    print("\nProcessing Requests:")
    priority_queue.dequeue()
    priority_queue.dequeue()
    priority_queue.display()
print("\n=====================================")
print("        PROGRAM COMPLETED")
print("=====================================")
```

```
=====================================
   LIBRARY BOOK REQUEST SYSTEM
=====================================

---- Normal Queue (FIFO) ----
Request added: Ravi (Student)
Request added: Dr. Kumar (Faculty)
Request added: Anita (Student)
Request added: Prof. Rao (Faculty)

Pending Requests (FIFO):
Ravi - Student - Python Basics
Dr. Kumar - Faculty - AI Research
Anita - Student - Data Structures
Prof. Rao - Faculty - Machine Learning

Processing Requests:
Request processed: Ravi
Request processed: Dr. Kumar

Pending Requests (FIFO):
Anita - Student - Data Structures
Prof. Rao - Faculty - Machine Learning

---- Priority Queue (Faculty First) ----
Request added: Ravi (Student)
Request added: Dr. Kumar (Faculty)
Request added: Anita (Student)
Request added: Prof. Rao (Faculty)

Pending Requests (Priority Queue):
Dr. Kumar - Faculty - AI Research
Prof. Rao - Faculty - Machine Learning
Ravi - Student - Python Basics
Anita - Student - Data Structures

Processing Requests:
Request processed: Dr. Kumar (Faculty)
Request processed: Prof. Rao (Faculty)

Pending Requests (Priority Queue):
                                                    Ln 268, Col 52    Spaces:
```

Task 2: Library Book Request System (Queue & Priority Queue)

Observation

The library request system was implemented using:

- Normal Queue (FIFO)
- Priority Queue

Observations include:

- In a normal queue, requests were processed in the order they arrived.
- In the priority queue, faculty requests were processed before student requests.
- FIFO ensures fairness, while priority queues improve service efficiency for important users.

Task 3: Emergency Help Desk (Stack Implementation)

Scenario

SR University's IT Help Desk receives technical support tickets from students and staff. While tickets are received sequentially, issue escalation follows a Last-In, First-Out (LIFO) approach.

CODE:

```
# ---------------------------------
# Stack Implementation - Help Desk Tickets
# ---------------------------------

class HelpDeskStack:
    def __init__(self, max_size=100):
        self.stack = []
        self.max_size = max_size

    def push(self, ticket):
        """Add a ticket to the stack"""
        if self.is_full():
            print("Stack is full! Cannot add more tickets.")
            return False
        self.stack.append(ticket)
        print(f"Ticket added: {ticket['id']} - {ticket['issue']}")
        return True
```

```
def pop(self):
    """Remove and return the top ticket (LIFO)"""
    if self.is_empty():
        print("Stack is empty! No tickets to resolve.")
        return None
    ticket = self.stack.pop()
    print(f"Ticket resolved: {ticket['id']} - {ticket['issue']}")
    return ticket

def peek(self):
    """View the top ticket without removing it"""
    if self.is_empty():
        print("Stack is empty!")
        return None
    return self.stack[-1]

def is_empty(self):
    """Check if stack is empty"""
    return len(self.stack) == 0

def is_full(self):
    """Check if stack is full"""
    return len(self.stack) >= self.max_size

def display(self):
    """Display all tickets in stack"""
    if self.is_empty():
        print("No pending tickets.")
        return
    print("\nPending Tickets (LIFO Stack):")
    for i, ticket in enumerate(reversed(self.stack)):
```

```
        print(f"  {i+1}. {ticket['id']} - {ticket['issue']} ({ticket['priority']})")


# ---------------------------------
# Help Desk Simulation
# ---------------------------------


if __name__ == "__main__":
    print("=====================================")
    print("   IT HELP DESK TICKET SYSTEM")
    print("=====================================")

    help_desk = HelpDeskStack()

    # Simulate 5 tickets
    print("\n---- Adding Tickets ----")
    help_desk.push({"id": "T001", "issue": "Login Error", "priority": "Low"})
    help_desk.push({"id": "T002", "issue": "Printer Not Working", "priority": "Medium"})
    help_desk.push({"id": "T003", "issue": "Network Outage", "priority": "High"})
    help_desk.push({"id": "T004", "issue": "Software Installation", "priority": "Low"})
    help_desk.push({"id": "T005", "issue": "Server Crash", "priority": "Critical"})

    help_desk.display()

    # Peek at top ticket
    print(f"\nCurrent ticket being addressed: {help_desk.peek()['id']}")

    # Resolve tickets (LIFO)
    print("\n---- Resolving Tickets (LIFO) ----")
    help_desk.pop()
    help_desk.pop()
```

```
    help_desk.display()


    # Check status
    print(f"\nStack empty: {help_desk.is_empty()}")
    print(f"Stack full: {help_desk.is_full()}")


print("\n======================================")
print("      HELP DESK SIMULATION ENDED")
print("======================================")
# ---------------------------------
# Hash Table Implementation
# ---------------------------------
```

OUTPUT:

```
======================================
======================================
   IT HELP DESK TICKET SYSTEM
======================================

---- Adding Tickets ----
Ticket added: T001 - Login Error
Ticket added: T002 - Printer Not Working
Ticket added: T003 - Network Outage
Ticket added: T004 - Software Installation
Ticket added: T005 - Server Crash

Pending Tickets (LIFO Stack):
  1. T005 - Server Crash (Critical)
  2. T004 - Software Installation (Low)
  3. T003 - Network Outage (High)
  4. T002 - Printer Not Working (Medium)
  5. T001 - Login Error (Low)

Current ticket being addressed: T005

---- Resolving Tickets (LIFO) ----
Ticket resolved: T005 - Server Crash
Ticket resolved: T004 - Software Installation

Pending Tickets (LIFO Stack):
  1. T003 - Network Outage (High)
  2. T002 - Printer Not Working (Medium)
  3. T001 - Login Error (Low)

Stack empty: False
Stack full: False
```

Task 3: Help Desk Ticket System (Stack Implementation)

Observation

A Help Desk Ticket System was implemented using the **Stack (LIFO)** principle.

Observations:

- Newly added tickets were placed at the top of the stack.
- The most recent ticket was resolved first.
- Push and pop operations executed efficiently in constant time.
- Stack behavior effectively simulated real-world emergency handling systems.

Task 4: Hash Table

Objective

To implement a Hash Table and understand collision handling.

Task Description

Use AI to generate a hash table with:

• Insert

• Search

• Delete

CODE:

```
class HashTable:
    def __init__(self, size=10):
        """Initialize hash table with chaining for collision handling"""
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        """Hash function to compute index"""
        return hash(key) % self.size

    def insert(self, key, value):
        """Insert key-value pair using chaining"""
        index = self._hash(key)
        # Check if key already exists and update
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index][i] = (key, value)
                print(f"Updated: {key} = {value}")
```

```
            return
    # Add new entry
    self.table[index].append((key, value))
    print(f"Inserted: {key} = {value}")

def search(self, key):
    """Search for value by key"""
    index = self._hash(key)
    for k, v in self.table[index]:
        if k == key:
            print(f"Found: {key} = {v}")
            return v
    print(f"Not found: {key}")
    return None

def delete(self, key):
    """Delete key-value pair"""
    index = self._hash(key)
    for i, (k, v) in enumerate(self.table[index]):
        if k == key:
            self.table[index].pop(i)
            print(f"Deleted: {key}")
            return True
    print(f"Not found: {key}")
    return False

def display(self):
    """Display all entries"""
    print("\nHash Table Contents:")
    for i, chain in enumerate(self.table):
        if chain:
```

```
            print(f"  Index {i}: {chain}")


# ---------------------------------
# Hash Table Testing
# ---------------------------------


if __name__ == "__main__":
    print("=====================================")
    print("      HASH TABLE IMPLEMENTATION")
    print("=====================================")

    ht = HashTable(5)

    print("\n---- Inserting Data ----")
    ht.insert("name", "Alice")
    ht.insert("age", 25)
    ht.insert("city", "New York")
    ht.insert("email", "alice@example.com")
    ht.insert("phone", "123-4567")  # Potential collision

    ht.display()

    print("\n---- Searching Data ----")
    ht.search("name")
    ht.search("age")
    ht.search("country")  # Not found

    print("\n---- Updating Data ----")
    ht.insert("age", 26)

    print("\n---- Deleting Data ----")
```

```
    ht.delete("email")

    ht.delete("country")  # Not found


    ht.display()


print("\n=====================================")

print("    HASH TABLE DEMO COMPLETED")

print("=====================================")
```

OUTPUT:

```
      HASH TABLE IMPLEMENTATION
====================================

---- Inserting Data ----
Inserted: name = Alice
Inserted: age = 25
Inserted: city = New York
Inserted: email = alice@example.com
Inserted: phone = 123-4567

Hash Table Contents:
  Index 1: [('city', 'New York'), ('phone', '123-4567')]
  Index 2: [('name', 'Alice'), ('age', 25)]
  Index 4: [('email', 'alice@example.com')]

---- Searching Data ----
Found: name = Alice
Found: age = 25
Not found: country

---- Updating Data ----
Updated: age = 26

---- Deleting Data ----
Deleted: email
Not found: country

Hash Table Contents:
  Index 1: [('city', 'New York'), ('phone', '123-4567')]
  Index 2: [('name', 'Alice'), ('age', 26)]
```

Task 4: Hash Table Implementation

Observation

A hash table using **chaining technique** was implemented for storing key–value pairs.

Observations:

- Hash function successfully mapped keys to indices.
- Collisions were handled using linked chaining.
- Searching, insertion, and deletion operations were performed efficiently

11.3 ASSIGNMENT

Task 5: Real-Time Application Challenge

Scenario

Design a Campus Resource Management System with the following features:

- Student Attendance Tracking
- Event Registration System
- Library Book Borrowing
- Bus Scheduling System
- Cafeteria Order Queue

CODE:

```
class AttendanceNode:
    def __init__(self, student_id, name, date, status):
        self.data = {"id": student_id, "name": name, "date": date, "status": status}
        self.next = None


class AttendanceTracker:
    def __init__(self):
        self.head = None


    def mark_attendance(self, student_id, name, date, status):
        """Mark attendance for a student"""
        new_node = AttendanceNode(student_id, name, date, status)
        if not self.head:
            self.head = new_node
        else:
            current = self.head
            while current.next:
                current = current.next
            current.next = new_node
        print(f"Attendance marked: {name} - {status}")
```

```
    def display_attendance(self):
        """Display all attendance records"""
        current = self.head
        if not current:
            print("No attendance records.")
            return
        print("\nAttendance Records:")
        while current:
            d = current.data
            print(f"  {d['id']} - {d['name']}: {d['status']} ({d['date']})")
            current = current.next

# ---------------------------------
# 2. Event Registration System (Hash Table)
# ---------------------------------

class EventRegistration:
    def __init__(self):
        self.events = {}

    def register_event(self, student_id, name, event):
        """Register student for event"""
        self.events[student_id] = {"name": name, "event": event}
        print(f"Registered: {name} for {event}")

    def find_registration(self, student_id):
        """Find registration by ID"""
        if student_id in self.events:
            return self.events[student_id]
        return None
```

```
    def display_registrations(self):
        """Display all registrations"""
        print("\nEvent Registrations:")
        for sid, data in self.events.items():
            print(f"  {sid}: {data['name']} - {data['event']}")

# ---------------------------------
# 3. Library Book Borrowing (Stack)
# ---------------------------------

class LibraryStack:
    def __init__(self):
        self.stack = []

    def borrow_book(self, student_id, book_title):
        """Borrow a book"""
        self.stack.append({"student_id": student_id, "book": book_title})
        print(f"Book borrowed: {book_title}")

    def return_book(self):
        """Return most recent borrowed book"""
        if not self.stack:
            print("No books to return.")
            return
        book = self.stack.pop()
        print(f"Book returned: {book['book']}")

    def display_borrowed(self):
        """Display borrowed books"""
        print("\nBorrowed Books (LIFO):")
        for b in reversed(self.stack):
```

```
        print(f"  {b['student_id']}: {b['book']}")

# ---------------------------------
# 4. Bus Scheduling System (Queue)
# ---------------------------------

class BusSchedule:
    def __init__(self):
        self.queue = []

    def schedule_bus(self, bus_id, route, time):
        """Schedule a bus"""
        self.queue.append({"id": bus_id, "route": route, "time": time})
        print(f"Bus scheduled: {bus_id} - {route} at {time}")

    def depart_bus(self):
        """Bus departs (FIFO)"""
        if not self.queue:
            print("No buses scheduled.")
            return
        bus = self.queue.pop(0)
        print(f"Bus departed: {bus['id']}")

    def display_schedule(self):
        """Display schedule"""
        print("\nBus Schedule (FIFO):")
        for b in self.queue:
            print(f"  {b['id']} → {b['route']} at {b['time']}")

# ---------------------------------
# 5. Cafeteria Order Queue (Priority Queue)
```

11.3 ASSIGNMENT

```
# ---------------------------------


class CafeteriaQueue:
    def __init__(self):
        self.queue = []

    def place_order(self, student_id, is_vip, order):
        """Place order with priority"""
        priority = 0 if is_vip else 1
        self.queue.append((priority, {"id": student_id, "order": order}))
        self.queue.sort(key=lambda x: x[0])
        print(f"Order placed: {student_id} - {order}")

    def serve_order(self):
        """Serve next order"""
        if not self.queue:
            print("No orders.")
            return
        p, order = self.queue.pop(0)
        print(f"Order served: {order['id']}")

    def display_orders(self):
        """Display pending orders"""
        print("\nPending Orders (Priority):")
        for p, o in self.queue:
            ptype = "VIP" if p == 0 else "Regular"
            print(f"  {ptype}: {o['id']} - {o['order']}")

# ---------------------------------
# Main Demo
# ---------------------------------
```

```
if __name__ == "__main__":
    print("="*50)
    print("CAMPUS RESOURCE MANAGEMENT SYSTEM")
    print("="*50)

    # Attendance
    print("\n--- Student Attendance Tracking ---")
    attendance = AttendanceTracker()
    attendance.mark_attendance("S001", "Alice", "2024-01-15", "Present")
    attendance.mark_attendance("S002", "Bob", "2024-01-15", "Absent")
    attendance.display_attendance()

    # Events
    print("\n--- Event Registration ---")
    events = EventRegistration()
    events.register_event("S001", "Alice", "Tech Summit")
    events.register_event("S002", "Bob", "Sports Day")
    events.display_registrations()

    # Library
    print("\n--- Library Book Borrowing ---")
    library = LibraryStack()
    library.borrow_book("S001", "Python Guide")
    library.borrow_book("S002", "Data Science")
    library.display_borrowed()
    library.return_book()
    library.display_borrowed()

    # Bus
    print("\n--- Bus Scheduling ---")
```

```
bus = BusSchedule()
bus.schedule_bus("B01", "Route A", "8:00 AM")
bus.schedule_bus("B02", "Route B", "8:30 AM")
bus.display_schedule()
bus.depart_bus()
bus.display_schedule()

# Cafeteria
print("\n--- Cafeteria Order Queue ---")
cafeteria = CafeteriaQueue()
cafeteria.place_order("S001", False, "Biryani")
cafeteria.place_order("S002", True, "Pizza")
cafeteria.place_order("S003", False, "Pasta")
cafeteria.display_orders()
cafeteria.serve_order()
cafeteria.display_orders()

print("\n" + "="*50)
print("SYSTEM DEMO COMPLETED")
print("="*50)
```

11.3 ASSIGNMENT

OUTPUT:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

==================================================
CAMPUS RESOURCE MANAGEMENT SYSTEM
==================================================

--- Student Attendance Tracking ---
Attendance marked: Alice - Present
Attendance marked: Bob - Absent

Attendance Records:
  S001 - Alice: Present (2024-01-15)
  S002 - Bob: Absent (2024-01-15)

--- Event Registration ---
Registered: Alice for Tech Summit
Registered: Bob for Sports Day

Event Registrations:
  S001: Alice - Tech Summit
  S002: Bob - Sports Day

--- Library Book Borrowing ---
Book borrowed: Python Guide
Book borrowed: Data Science

Borrowed Books (LIFO):
  S002: Data Science
  S001: Python Guide
Book returned: Data Science

Borrowed Books (LIFO):
  S001: Python Guide

--- Bus Scheduling ---
Bus scheduled: B01 - Route A at 8:00 AM
Bus scheduled: B02 - Route B at 8:30 AM
```